

**LAPORAN PROJEK AKHIR SEMESTER 5
PENGOLAHAN CITRA IF-7A**



Klasifikasi Gambar Kucing vs Anjing Menggunakan CNN

Disusun Oleh:

Nama: Muhammad Zulfri Choiril Anwar (230411100176)

Nama: Muhammad Ridho Faiz (230411100197)

Dosen Pengampu:

Nama: Dr. Cucun Very Angkoso, S.T., MT

NIP: 197802252005011001

**PRODI TEKNIK INFORMATIKA
JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS TRUNOJOYO MADURA**

2025

DAFTAR ISI

ABSTRAK.....	iv
ABSTRACT	v
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah	1
1.3 Tujuan Proyek.....	1
1.4 Batasan Proyek.....	1
BAB II BUSINESS UNDERSTANDING	2
2.1 Konteks Permasalahan.....	2
2.2 Tujuan Bisnis.....	2
2.3 Tujuan Machine Learning	2
2.4 Kriteria Keberhasilan.....	2
BAB III DATA UNDERSTANDING	3
3.1 Sumber Data	3
3.2 Struktur Fitur dan Label.....	3
3.3 Eksplorasi Data (EDA).....	3
3.4 Permasalahan Data.....	3
BAB IV DATA PREPARATION	4
4.1 Pembersihan Data (Handling Corrupt Images)	4
4.2 Transformasi Citra (Image Transformation).....	5
4.3 Pembagian Data (Splitting)	6
4.4 Dataset Final.....	6
BAB V MODELING.....	7
5.1 Pemilihan Algoritma	7
5.2 Arsitektur Model (Network Architecture).....	7
5.3 Skema Pelatihan.....	9
BAB VI EVALUATION	10
6.1 Evaluasi.....	10
6.2 Pengujian Kualitatif (Real-world Testing).....	10
6.3 Analisis Hasil	11
6.4 Model Final.....	11
BAB VII KESIMPULAN DAN SARAN	12

7.1 Kesimpulan	12
7.2 Saran Pengembangan	12

ABSTRAK

Latar belakang proyek ini adalah kebutuhan akan pengenalan pola otomatis pada citra digital yang semakin masif. Tujuan proyek adalah membangun model *Deep Learning* untuk mengklasifikasikan gambar hewan peliharaan menjadi dua kelas: Anjing dan Kucing. Dataset yang digunakan bersumber dari folder PetImages yang berisi total 24.997 gambar RGB. Metode yang diterapkan adalah *Convolutional Neural Network* (CNN) menggunakan framework PyTorch, dengan tahapan *preprocessing* meliputi *resize*, normalisasi, dan penanganan file korup. Ringkasan hasil menunjukkan model mampu memberikan prediksi kelas dengan tingkat keyakinan (*confidence score*) tertentu pada data uji.

ABSTRACT

The background of this project is the growing need for automated pattern recognition in digital images. The objective of this project is to develop a Deep Learning model to classify pet images into two classes: Dogs and Cats. The dataset used is sourced from the PetImages folder, which contains a total of 24,997 RGB images. The method applied is a Convolutional Neural Network (CNN) using the PyTorch framework, with preprocessing stages including image resizing, normalization, and handling of corrupted files. The results indicate that the model is able to provide class predictions with a certain level of confidence (confidence score) on the test data.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital, data visual seperti gambar dan video tumbuh secara eksponensial. Kemampuan komputer untuk "melihat" dan membedakan objek (*Computer Vision*) menjadi krusial untuk berbagai aplikasi, mulai dari manajemen galeri foto otomatis hingga sistem keamanan. Oleh karena itu, dibutuhkan model otomatis yang dapat membedakan objek dasar, seperti anjing dan kucing, sebagai fondasi untuk sistem klasifikasi yang lebih kompleks.

1.2 Rumusan Masalah

- Bagaimana membangun arsitektur *Convolutional Neural Network* (CNN) untuk membedakan gambar anjing dan kucing?
- Bagaimana menangani permasalahan kualitas data gambar (file korup, ukuran berbeda) agar dapat diproses oleh model?

1.3 Tujuan Proyek

- Mengklasifikasikan citra digital menjadi dua kelas label: *Dog* (Anjing) atau *Cat* (Kucing).
- Mengevaluasi kinerja model CNN yang dibangun menggunakan PyTorch dalam memprediksi data baru.

1.4 Batasan Proyek

- Dataset terbatas pada folder PetImages (kelas Cat dan Dog).
- Gambar input diubah ukurannya (resize) menjadi dimensi 128x128 piksel.
- Framework yang digunakan adalah PyTorch dengan akselerasi GPU (CUDA) jika tersedia.
- Model fokus pada arsitektur CNN (bukan metode tradisional seperti SVM).

BAB II

BUSINESS UNDERSTANDING

2.1 Konteks Permasalahan

Kesulitan dalam menyortir ribuan foto hewan peliharaan secara manual membutuhkan waktu dan tenaga yang besar. Sistem otomatis diperlukan untuk mengelompokkan foto berdasarkan jenis hewan.

2.2 Tujuan Bisnis

Menghasilkan purwarupa (*prototype*) sistem cerdas yang dapat membantu pengguna atau platform media sosial dalam mengategorikan konten gambar hewan peliharaan secara otomatis dan efisien.

2.3 Tujuan Machine Learning

Mengembangkan model *Supervised Learning* (CNN) yang mampu meminimalkan *loss* (kesalahan prediksi) dan memaksimalkan akurasi pada data validasi dalam membedakan anjing dan kucing.

2.4 Kriteria Keberhasilan

- Model dapat berjalan tanpa error pada data gambar yang valid.
- Mampu mendeteksi dan memfilter gambar yang rusak (*corrupt*) sebelum pelatihan.
- Akurasi prediksi pada data uji coba acak (misal: *alpha-dog.jpg*) menunjukkan label yang benar dengan keyakinan tinggi.

BAB III

DATA UNDERSTANDING

3.1 Sumber Data

Data yang digunakan berasal dari dataset *Dog and Cat Classification Dataset* yang tersedia di Kaggle dan dapat diakses melalui tautan berikut:

<https://www.kaggle.com/datasets/bhavikjikadara/dog-and-cat-classification-dataset>

Dataset ini terdiri dari dua sub-folder utama berdasarkan label kelasnya, yaitu **Cat** dan **Dog**, yang masing-masing berisi citra kucing dan anjing untuk keperluan klasifikasi.

3.2 Struktur Fitur dan Label

- **Fitur:** Matriks piksel gambar (citra digital) yang dikonversi menjadi Tensor RGB.
- **Label:** Biner, yaitu 0 untuk Cat dan 1 untuk Dog.

3.3 Eksplorasi Data (EDA)

- **Jumlah Data:** Total gambar yang terbaca adalah 24.997 file.
- **Pembagian Data:** Dataset dibagi menjadi Data Latih (80% atau 19.997 gambar) dan Data Validasi (20% atau 5.000 gambar).
- **Visualisasi:** Data mencakup berbagai ras anjing dan kucing dengan variasi pencahayaan dan pose.

3.4 Permasalahan Data

- **File Korup/Rusak:** Ditemukan file gambar yang tidak dapat dibaca atau truncated (contoh: 9041.jpg pada folder Dog) yang kemudian dikarantina ke folder ./CorruptImages.
- **Variasi Ukuran:** Ukuran asli gambar bervariasi, sehingga perlu diseragamkan menjadi 128x128 piksel.
- **Format Warna:** Beberapa gambar mungkin grayscale atau CMYK, yang kemudian dipaksa konversi ke RGB melalui fungsi `transforms.Lambda(lambda x: x.convert("RGB"))`.

BAB IV

DATA PREPARATION

4.1 Pembersihan Data (Handling Corrupt Images)

Sebelum masuk ke model, data dibersihkan dari file yang rusak agar tidak menghentikan proses *training*. File yang tidak valid dipindahkan atau diabaikan.

```
try:
    with warnings.catch_warnings(record=True) as w:
        warnings.simplefilter("error", UserWarning)
        warnings.simplefilter("error", Image.DecompressionBombWarning)

        with Image.open(file_path) as img:
            img.verify()
            with Image.open(file_path) as img2:
                img2.load()

except (IOError, SyntaxError, OSError, UserWarning, Warning) as e:
    is_corrupt = True
    error_msg = str(e)

    if is_corrupt:
        dst_path = os.path.join(dst_folder, filename)
        print(f"❌ Isu ditemukan ({error_msg}) -> Memindahkan: {filename}")
```

4.2 Transformasi Citra (Image Transformation)

- **Resize:** Menyamakan semua gambar menjadi 128x128 piksel.
- **To Tensor:** Mengubah array gambar menjadi format Tensor PyTorch (C, H, W) dengan nilai 0.0 - 1.0.
- **Normalization:** (Opsional) Menstandarisasi nilai piksel.

```
BATCH_SIZE = 32
IMG_SIZE = (128, 128)

train_transform = transforms.Compose([
    # Paksa convert ke RGB agar tidak crash di gambar Grayscale
    transforms.Lambda(lambda x: x.convert("RGB")),

    # Augmentasi Data (Mempersulit model belajar)
    transforms.Resize(IMG_SIZE),
    transforms.RandomHorizontalFlip(p=0.5), # Kadang dibalik kiri-kanan
    transforms.RandomRotation(degrees=15), # Kadang dimiringkan dikit
    transforms.ColorJitter(brightness=0.2, contrast=0.2), # Ubah pencahayaan

    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

# B. Transformasi Validasi (Bersih & Standar)
val_transform = transforms.Compose([
    transforms.Lambda(lambda x: x.convert("RGB")), # Tetap convert RGB
    transforms.Resize(IMG_SIZE), # Resize saja
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])
```

4.3 Pembagian Data (Splitting)

Dataset dibagi menjadi data latih (*training*) dan data validasi (*validation*)

- **Rasio:** 80% Training (± 20.000 gambar), 20% Validation (± 5.000 gambar).

```
raw_dataset = datasets.ImageFolder(root=DATASET_PATH)

total_count = len(raw_dataset)
train_count = int(0.8 * total_count)
val_count = total_count - train_count

train_subset, val_subset = random_split(raw_dataset, [train_count, val_count])

class DatasetFromSubset(Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform

    def __getitem__(self, index):
        x, y = self.subset[index] # Ambil gambar raw
        if self.transform:
            x = self.transform(x) # Terapkan transform khusus (Train/Val)
        return x, y

    def __len__(self):
        return len(self.subset)

train_data = DatasetFromSubset(train_subset, transform=train_transform)
val_data = DatasetFromSubset(val_subset, transform=val_transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True,
                           num_workers=0)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=0)
```

4.4 Dataset Final

Dimensi input model adalah [Batch_Size, 3, 128, 128] (3 channel warna RGB).

BAB V

MODELING

5.1 Pemilihan Algoritma

Alasan memilih **Convolutional Neural Network (CNN)** dibandingkan algoritma tradisional (seperti SVM/MLP):

- **Feature Extraction Otomatis:** CNN mampu mempelajari fitur visual (tepi, bentuk, tekstur) melalui layer konvolusi tanpa perlu ekstraksi fitur manual (seperti HOG atau SIFT).
- **Spatial Hierarchy:** Mempertahankan struktur spasial gambar.
- **Parameter Efficiency:** *Weight sharing* pada konvolusi membuat model lebih efisien dibanding Fully Connected Network biasa untuk data gambar.

5.2 Arsitektur Model (Network Architecture)

Model dibangun menggunakan `torch.nn.Module`.

- **Convolutional Layers:** Untuk mengekstrak fitur.
- **Pooling Layers (MaxPool):** Untuk mereduksi dimensi spasial dan mengambil fitur dominan.
- **Fully Connected Layers (Linear):** Untuk klasifikasi akhir menjadi 2 kelas.
- **Activation Function:** ReLU (Rectified Linear Unit) untuk non-linearitas.

```
class CatDogCNN(nn.Module):
    def __init__(self):
        super(CatDogCNN, self).__init__()

        # --- BLOK 1: Ekstraksi Fitur Awal (Detail Kasar: Garis, Sudut) ---
        # Input: 3 channel (RGB) -> Output: 32 filter
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3,
padding=1)
        self.bn1 = nn.BatchNorm2d(32) # Penstabil
```

```

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # Perkecil ukuran
gambar (128->64)

# --- BLOK 2: Ekstraksi Fitur Menengah (Bentuk: Mata, Telinga) ---
# Input: 32 -> Output: 64
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.bn2 = nn.BatchNorm2d(64)
# MaxPool lagi (64->32)

# --- BLOK 3: Ekstraksi Fitur Kompleks (Wajah utuh) ---
# Input: 64 -> Output: 128
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.bn3 = nn.BatchNorm2d(128)
# MaxPool lagi (32->16)

# --- BLOK 4: Deep Features (Abstrak) ---
# Input: 128 -> Output: 128 (Kita tahan di 128 biar hemat VRAM)
self.conv4 = nn.Conv2d(128, 128, kernel_size=3, padding=1)
self.bn4 = nn.BatchNorm2d(128)
# MaxPool lagi (16->8)

# --- CLASSIFIER (Otak Pengambil Keputusan) ---
# Gambar sekarang ukurannya 8x8 dengan tebal 128 layer
# Flatten:  $128 * 8 * 8 = 8192$  neuron
self.fc1 = nn.Linear(128 * 8 * 8, 512)
self.dropout = nn.Dropout(0.5) # KUNCI ANTI-OVERFITTING (Matikan 50%
neuron acak)
self.fc2 = nn.Linear(512, 2) # Output: 2 (Cat atau Dog)

def forward(self, x):
    # Alur Maju (Forward Pass)

    # Blok 1
    x = self.pool(F.relu(self.bn1(self.conv1(x))))

    # Blok 2

```

```

x = self.pool(F.relu(self.bn2(self.conv2(x))))

# Blok 3
x = self.pool(F.relu(self.bn3(self.conv3(x))))

# Blok 4
x = self.pool(F.relu(self.bn4(self.conv4(x))))

# Ratakan (Flatten) dari gambar 3D ke vektor 1D
x = x.view(-1, 128 * 8 * 8)

x = F.relu(self.fc1(x))
x = self.dropout(x) # Terapkan Dropout di sini
x = self.fc2(x)

return x
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CatDogCNN().to(device)

```

5.3 Skema Pelatihan

- **Loss Function:** CrossEntropyLoss (cocok untuk klasifikasi).
- **Optimizer:** Adam atau SGD (untuk update bobot).
- **Epoch:** Jumlah iterasi seluruh dataset.

BAB VI

EVALUATION

6.1 Evaluasi

Evaluasi dilakukan dengan menghitung akurasi prediksi pada data validasi/test.

- **Accuracy:** Persentase prediksi benar dari total data uji.
- **Loss:** Nilai error yang harus menurun seiring pelatihan.

6.2 Pengujian Kualitatif (Real-world Testing)

Model diuji coba menggunakan gambar acak dari internet atau folder tes khusus untuk melihat performa pada data "liar".

```
def prediksi_gambar(image_path):  
    # 1. Pastikan file ada  
    if not os.path.exists(image_path):  
        print(f"✗ File tidak ditemukan: {image_path}")  
        return  
  
    # 2. Buka Gambar  
    img = Image.open(image_path)  
  
    # 3. Preprocessing (HARUS SAMA PERSIS dengan Validation Transform)  
    #     Resize -> Convert RGB -> ToTensor -> Normalize  
    transform_prediksi = val_transform  
  
    # Tambahkan dimensi batch (karena model butuh [1, 3, 128, 128])  
    img_tensor = transform_prediksi(img).unsqueeze(0).to(device)  
  
    # 4. Prediksi  
    model.eval() # Mode evaluasi (matikan dropout)  
    with torch.no_grad():  
        outputs = model(img_tensor)
```

```

# Hitung probabilitas (keyakinan)
probs = torch.nn.functional.softmax(outputs, dim=1)
conf, predicted = torch.max(probs, 1)

# 5. Tampilkan Hasil
pred_label = class_names[predicted.item()]
confidence = conf.item() * 100

print(f"-----")
print(f"📁 File      : {image_path}")
print(f"🖼️ Prediksi  : {pred_label}")
print(f"📊 Keyakinan : {confidence:.2f}%")

```

6.3 Analisis Hasil

- Model berhasil memprediksi gambar alpha-dog.jpg sebagai *Dog* dengan keyakinan 77.67%.
- Jika terjadi kesalahan (misal kucing diprediksi anjing), hal ini mungkin disebabkan oleh *background* yang kompleks atau pose hewan yang tidak umum dalam data latih.

6.4 Model Final

Model dengan bobot (*weights*) terakhir atau yang memiliki loss validasi terendah disimpan untuk digunakan

BAB VII

KESIMPULAN DAN SARAN

7.1 Kesimpulan

- Model CNN berhasil dibangun menggunakan PyTorch untuk klasifikasi biner Anjing vs Kucing.
- Proses preprocessing seperti resize dan penanganan corrupt files sangat krusial untuk mencegah error saat pelatihan.
- Model mampu memberikan prediksi probabilitas (confidence score), tidak hanya label kelas semata.

7.2 Saran Pengembangan

- **Data Augmentation:** Menambahkan variasi data (rotasi, flip, zoom) untuk mencegah overfitting.
- **Transfer Learning:** Menggunakan model pre-trained seperti ResNet18 atau VGG16 yang sudah dilatih pada ImageNet untuk akurasi yang jauh lebih tinggi dengan waktu latih lebih singkat.
- **Hyperparameter Tuning:** Mencoba variasi learning rate, batch size, atau jumlah layer konvolusi untuk hasil optimal.